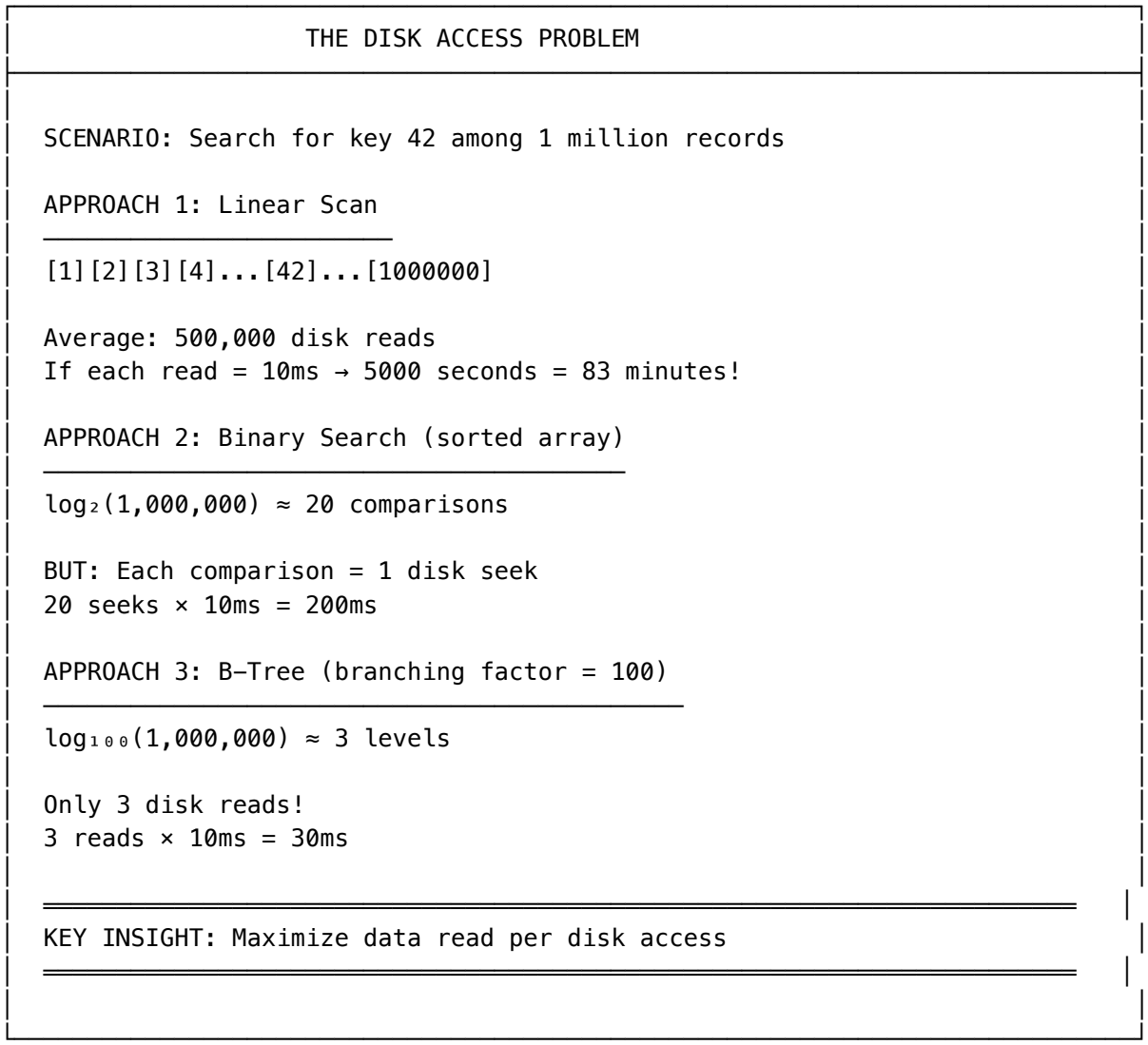
Chapter 2: B-Tree Basics - The Backbone of Database Indexes

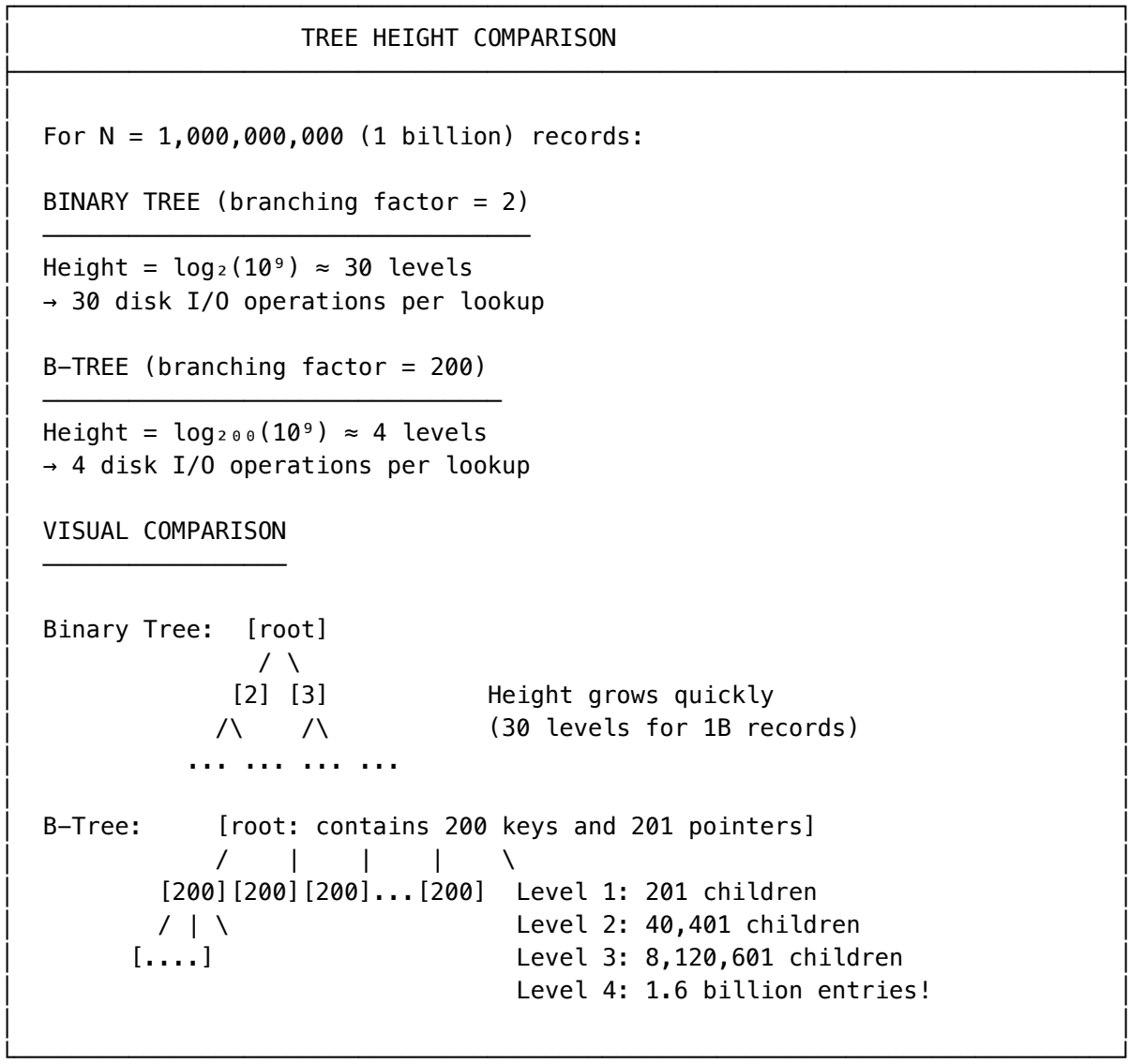
Table of Contents

Why B-Trees?

B-Trees are the **dominant data structure** for database indexes. Understanding why requires examining the fundamental problem: efficient disk access.

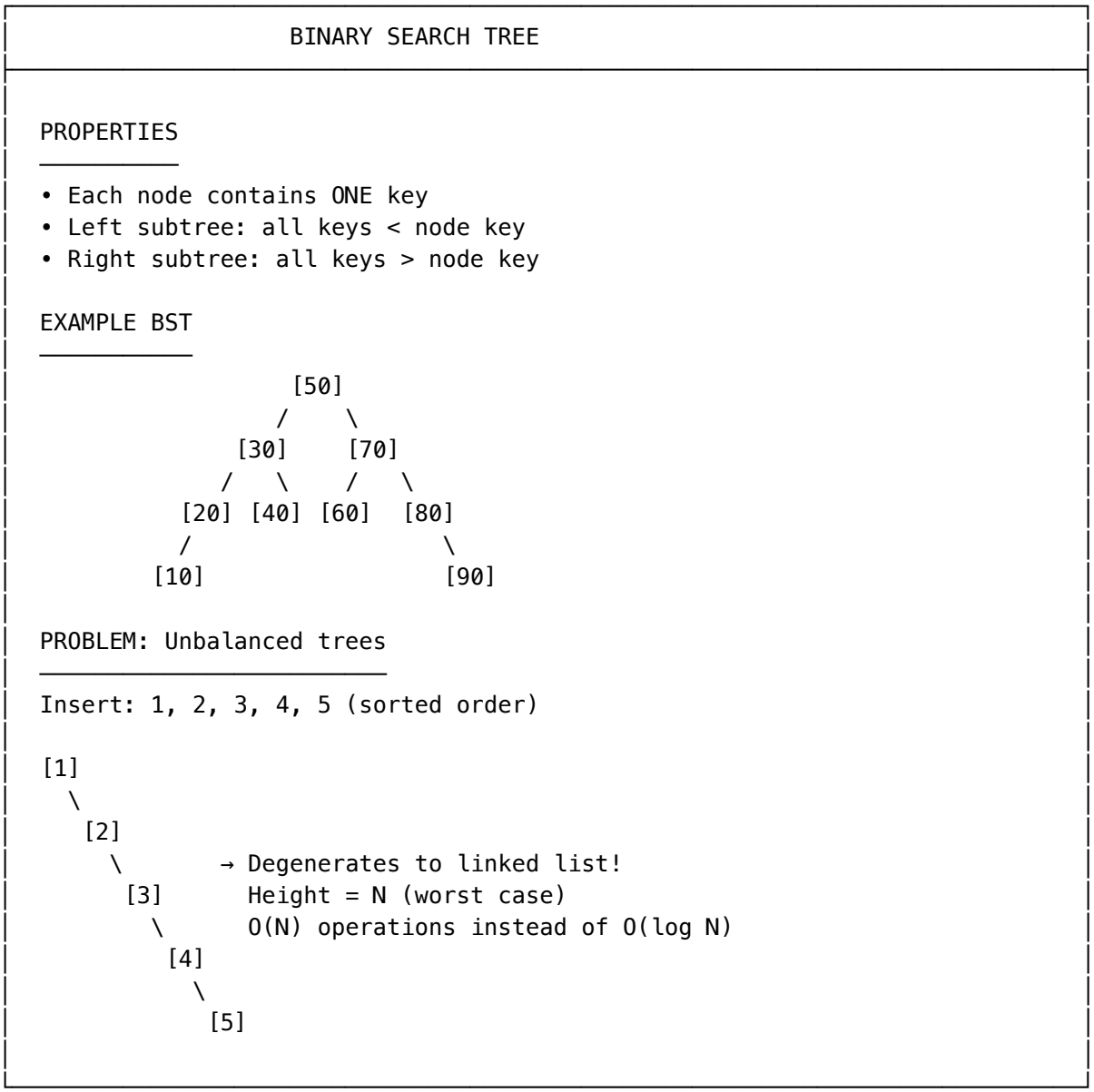


The High Branching Factor Advantage



From Binary Search Trees to B-Trees

Binary Search Tree (BST) Review



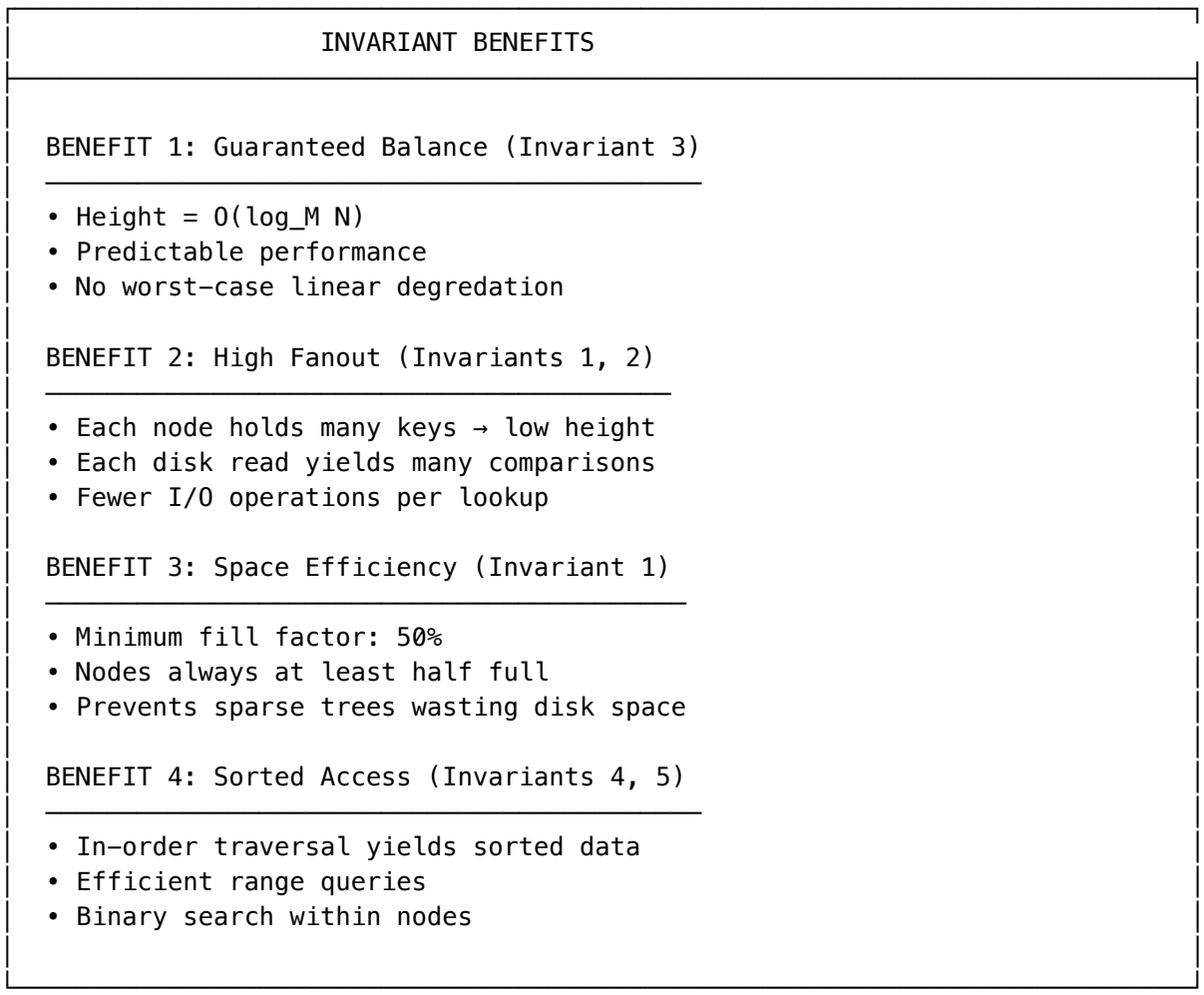
Self-Balancing BSTs (AVL, Red-Black)

```
┌──────────────────────────────────────────────────────────────────────┐
│                    SELF-BALANCING BSTs                               │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│  AVL TREE                                                            │
│  ────────                                                            │
│  • Balance factor = height(left) − height(right)                     │
│  • Invariant: |balance factor| ≤ 1 for every node                    │
│  • Rotations to rebalance after insert/delete                        │
│                                                                      │
│  RED-BLACK TREE                                                      │
│  ──────────────                                                      │
│  • Each node is Red or Black                                         │
│  • Root is always Black                                              │
│  • No two Red nodes in a row (parent-child)                          │
│  • Every path from root to NULL has same # of Black nodes            │
│                                                                      │
│  WHY NOT USE THESE FOR DATABASES?                                    │
│  ────────────────────────────────                                    │
│                                                                      │
│  Problem 1: Low branching factor                                     │
│  • Still binary (2 children per node)                                │
│  • log₂(N) height → too many disk reads                              │
│                                                                      │
│  Problem 2: Each node = 1 disk page?                                 │
│  • If page = 4KB and key = 8 bytes                                   │
│  • Utilization = 8 / 4096 = 0.2% (horrible!)                         │
│                                                                      │
│  Problem 3: Rebalancing locality                                     │
│  • Rotations may access distant nodes                                │
│  • Many disk seeks for single operation                              │
│                                                                      │
└──────────────────────────────────────────────────────────────────────┘
```

B-Tree Properties and Invariants

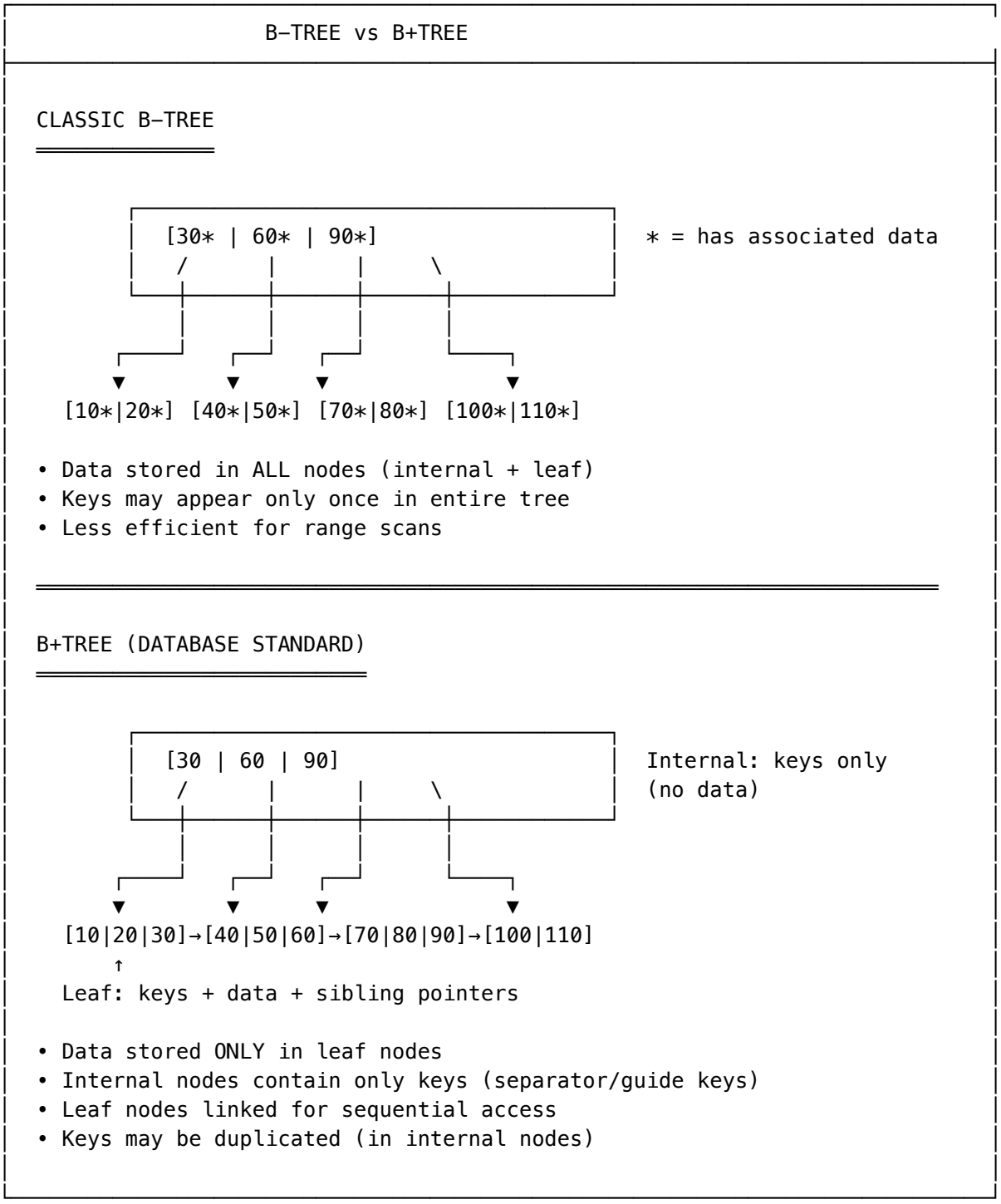
```
┌──────────────────────────────────────────────────────────────────────────────┐
│                    B-TREE FORMAL DEFINITION                                  │
├──────────────────────────────────────────────────────────────────────────────┤
│                                                                              │
│  A B-Tree of order M (or degree M) satisfies:                                │
│                                                                              │
│  INVARIANT 1: Node Key Limits                                                │
│  ─────────────────────────────                                               │
│  • Root: 1 to M-1 keys                                                       │
│  • Internal nodes: ⌈M/2⌉-1 to M-1 keys                                       │
│  • Leaf nodes: ⌈M/2⌉-1 to M-1 keys                                           │
│                                                                              │
│  INVARIANT 2: Child Pointer Count                                            │
│  ─────────────────────────────────                                           │
│  • If a node has K keys, it has K+1 children (for internal nodes)            │
│  • Leaf nodes have no children                                               │
│                                                                              │
│  INVARIANT 3: All Leaves at Same Depth                                       │
│  ──────────────────────────────────────                                      │
│  • Tree is perfectly balanced                                                │
│  • Every path from root to leaf has same length                              │
│                                                                              │
│  INVARIANT 4: Keys are Sorted Within Node                                    │
│  ─────────────────────────────────────────                                   │
│  • K₁ < K₂ < K₃ < ... < K□                                                   │
│                                                                              │
│  INVARIANT 5: Subtree Ordering                                               │
│  ─────────────────────────────                                               │
│  • All keys in subtree pointed by Pᵢ are:                                    │
│    - Greater than Kᵢ₋₁ (if i > 0)                                            │
│    - Less than Kᵢ (if i < number of keys)                                    │
│                                                                              │
│  EXAMPLE: B-Tree of Order 5 (M=5)                                            │
│  ─────────────────────────────────                                           │
│  • Each node: 2 to 4 keys (⌈5/2⌉-1 = 2, M-1 = 4)                             │
│  • Each internal node: 3 to 5 children                                       │
│                                                                              │
│          ┌──────────────────────────────┐                                    │
│          │     [30 | 60 | -- | --]      │                                    │
│          │      /      |       \        │                                    │
│          └──────┼──────┼────────┼───────┘                                    │
│                 │      │        │                                            │
│        ┌────────┘      │        └────────┐                                   │
│        ▼               ▼                 ▼                                   │
│  [10|20|--|--]   [40|50|--|--]    [70|80|90|--]                              │
│                                                                              │
└──────────────────────────────────────────────────────────────────────────────┘
```

Why These Invariants Matter



B+Tree: The Database Variant

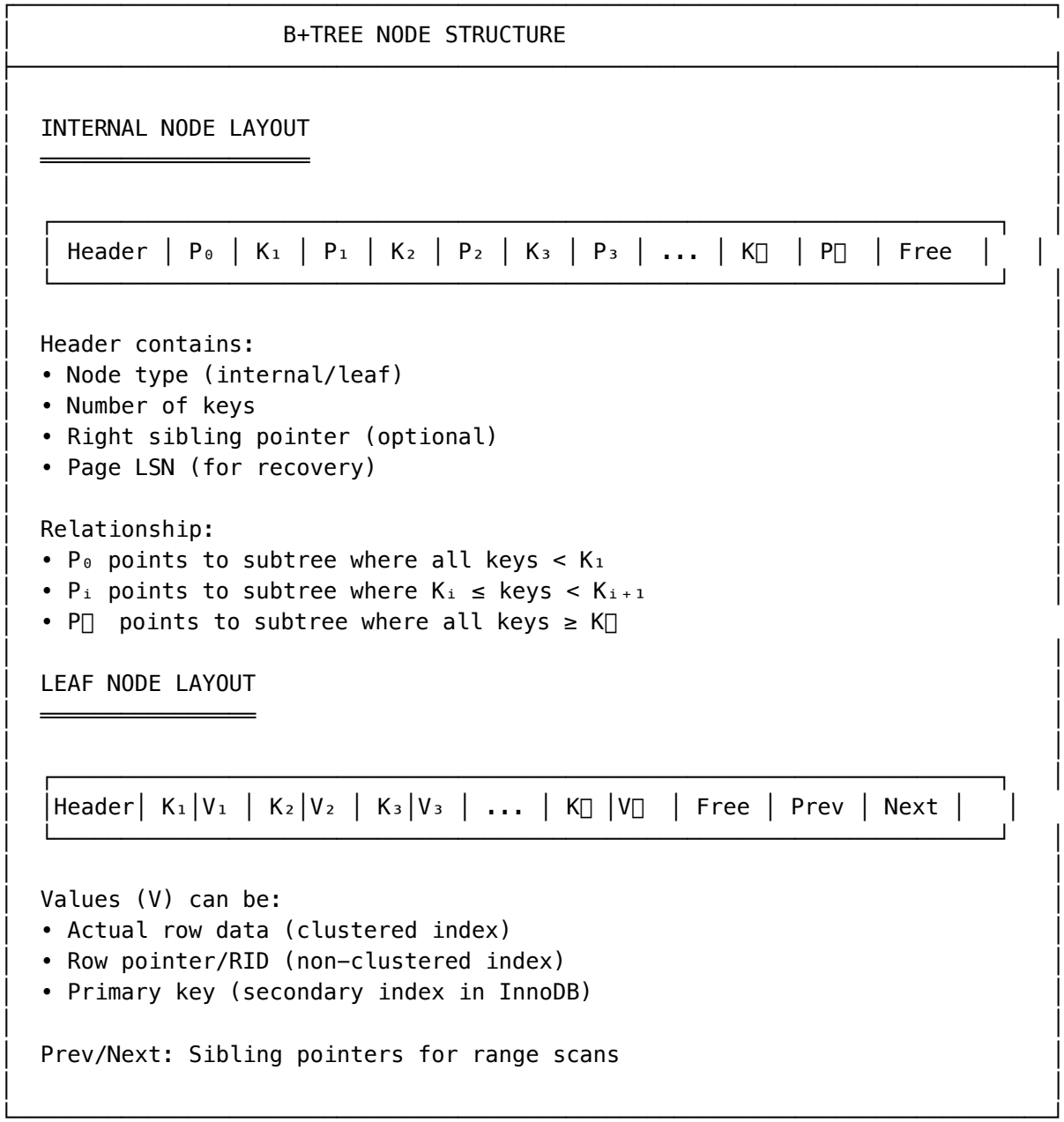
Most databases use B+Trees rather than classic B-Trees. Here's why:



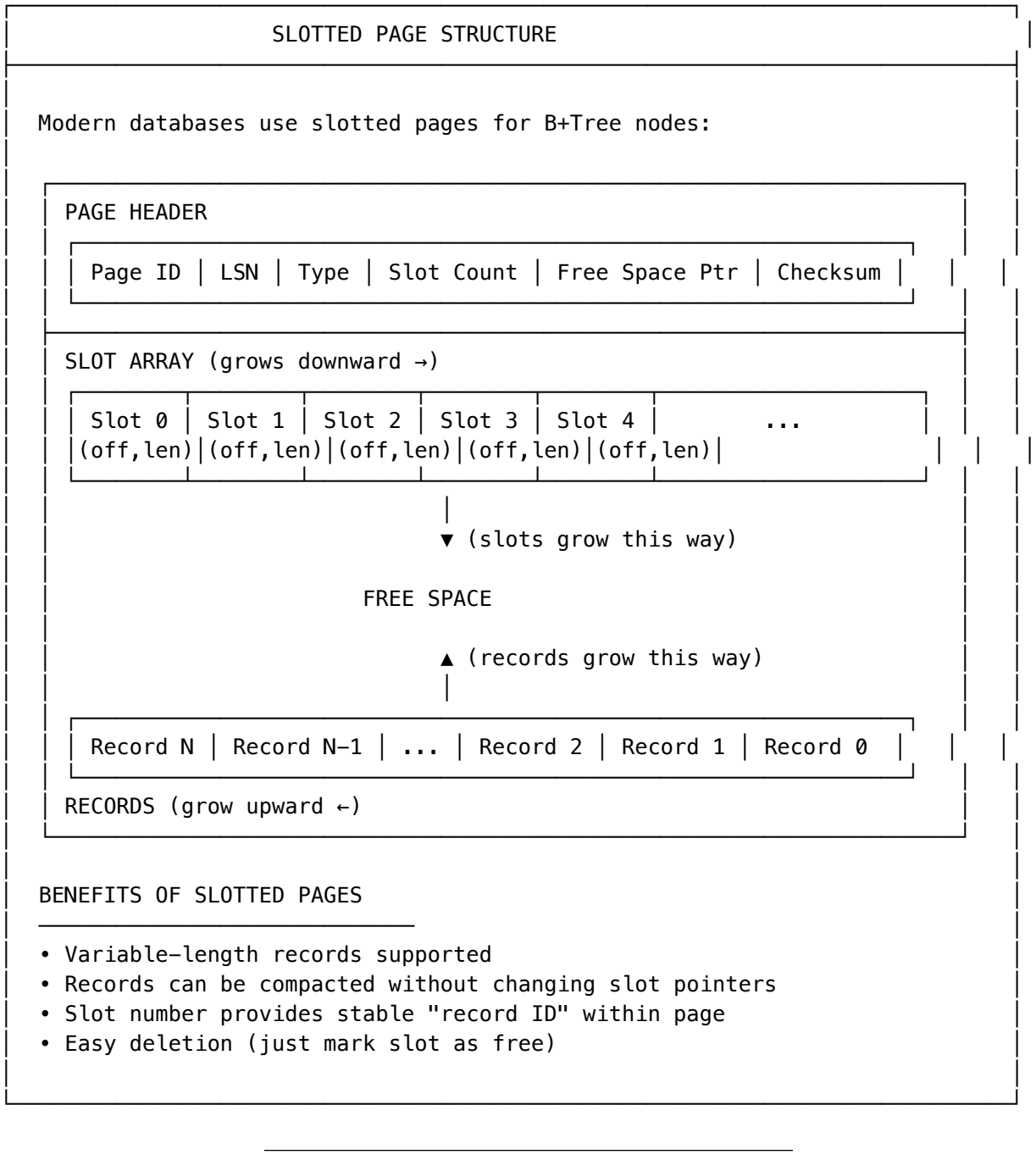
B+Tree Advantages for Databases

```
┌──────────────────────────────────────────────────────────────────────────┐
│                    WHY DATABASES USE B+TREES                             │
└──────────────────────────────────────────────────────────────────────────┘

ADVANTAGE 1: Higher Fanout in Internal Nodes
─────────────────────────────────────────────

B-Tree internal node:    [Key₁|Data₁|Ptr₁|Key₂|Data₂|Ptr₂|...]
B+Tree internal node:    [Key₁|Ptr₁|Key₂|Ptr₂|Key₃|Ptr₃|...]

If page = 8KB, key = 8 bytes, data = 100 bytes, pointer = 8 bytes:
• B-Tree: 8192 / (8 + 100 + 8) ≈ 70 keys per node
• B+Tree: 8192 / (8 + 8) ≈ 500 keys per node

→ B+Tree has 7x higher fanout → fewer levels → fewer disk reads

ADVANTAGE 2: Efficient Range Scans
──────────────────────────────────

Query: SELECT * FROM users WHERE age BETWEEN 25 AND 35

B+Tree approach:
1. Find leaf with age=25 (log N lookups)
2. Follow sibling pointers: [25]→[26]→[27]→...→[35]
3. Sequential scan through linked leaves

┌──────┐   ┌──────┐   ┌──────┐   ┌──────┐   ┌──────┐
│ 20   │──▶│ 25   │──▶│ 28   │──▶│ 32   │──▶│ 40   │
│ 22   │   │ 26   │   │ 30   │   │ 35   │   │ 45   │
│ 24   │   │ 27   │   │ 31   │   │ 38   │   │ 50   │
└──────┘   └──────┘   └──────┘   └──────┘   └──────┘
             ▲                     ▲
             │                     │
          start=25              end=35
             └─────────────────────┘
                Sequential scan

ADVANTAGE 3: Consistent Performance
───────────────────────────────────

• All data at same depth (leaf level)
• Every lookup traverses same number of levels
• Predictable latency

ADVANTAGE 4: Simpler Caching Strategy
─────────────────────────────────────

• Internal nodes are smaller → more fit in cache
• Cache internal nodes (hot), fetch leaves on demand
• Better memory utilization
```

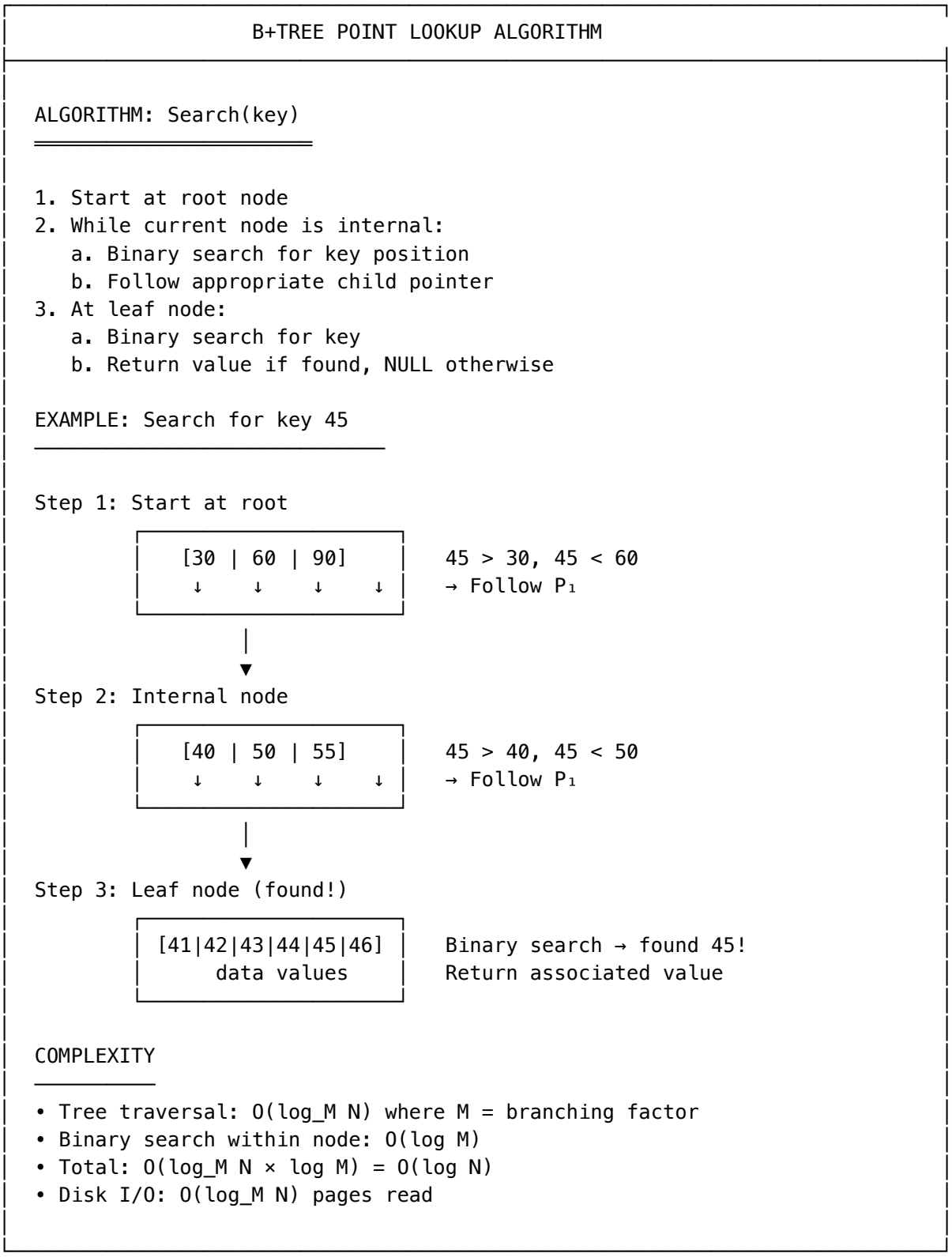
Node Structure and Layout



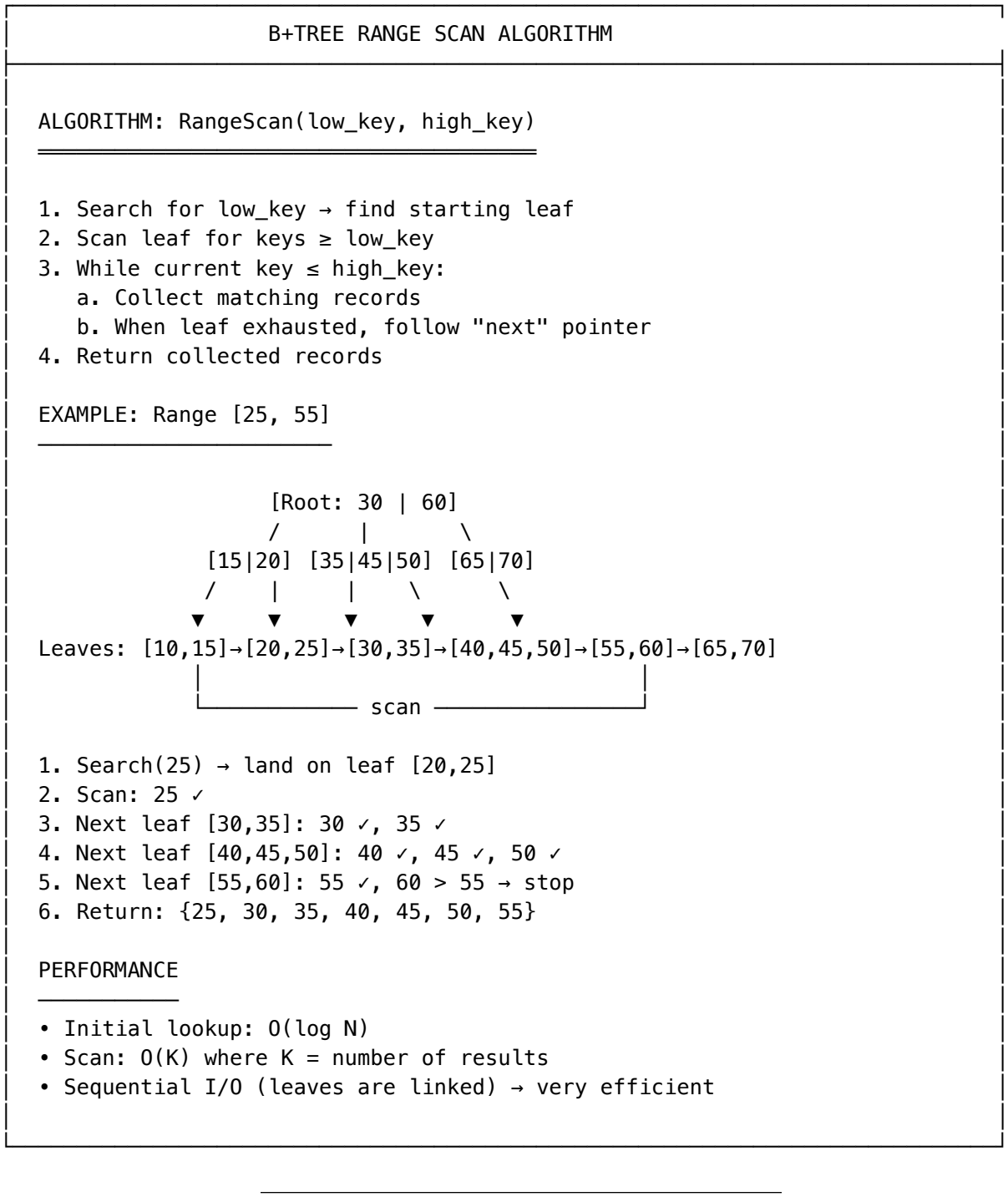
Slotted Page Organization



Lookup Operations



Range Query Algorithm



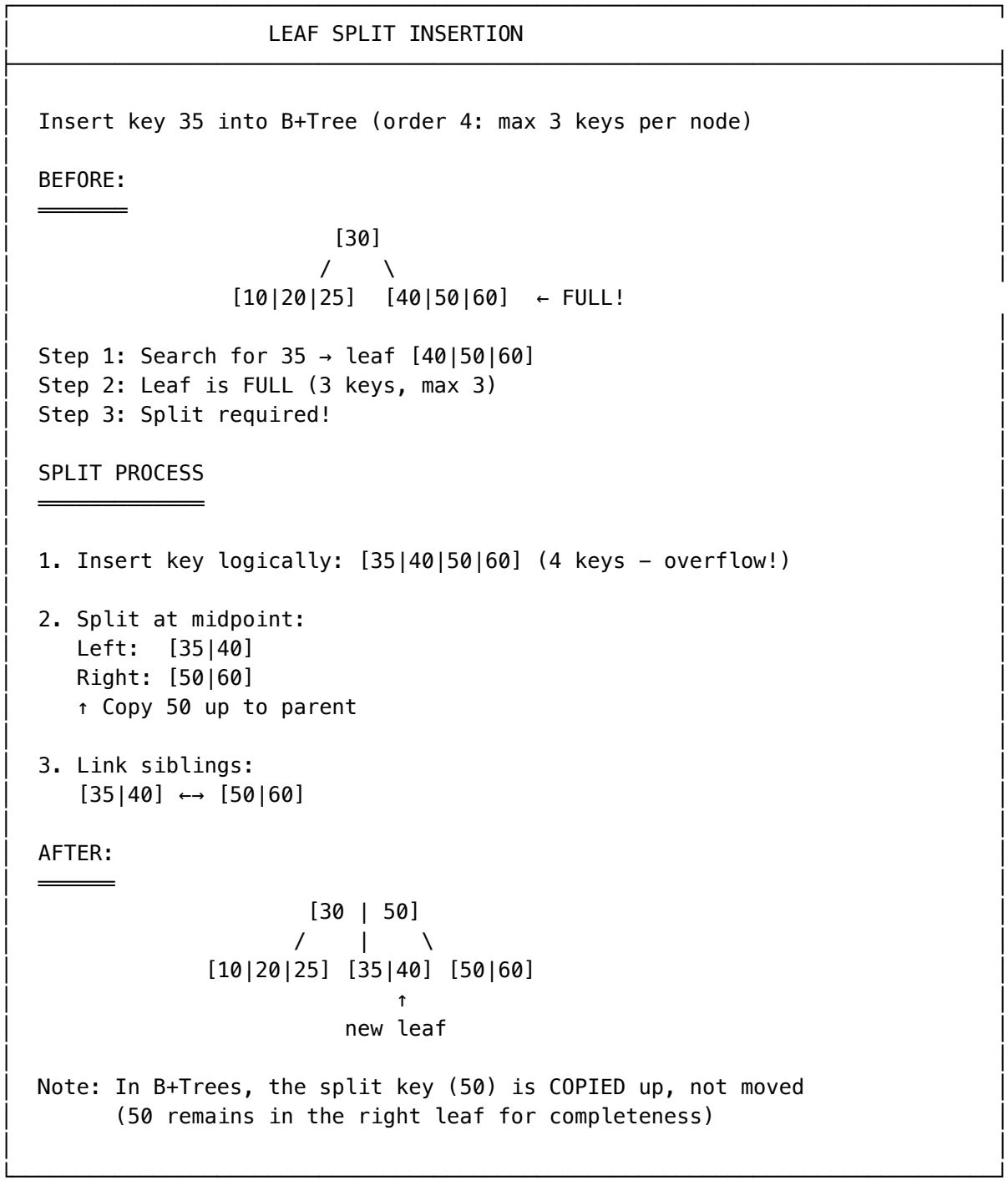
Insertion Operations

```
┌──────────────────────────────────────────────────────────────────────────┐
│                    B+TREE INSERTION ALGORITHM                            │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
│  ALGORITHM: Insert(key, value)                                           │
│  ═══════════════════════════════                                         │
│                                                                          │
│  1. Search for appropriate leaf node                                     │
│  2. If leaf has space:                                                   │
│     a. Insert key-value pair in sorted order                             │
│     b. Done!                                                             │
│  3. If leaf is full (overflow):                                          │
│     a. Split leaf into two nodes                                         │
│     b. Distribute keys evenly                                            │
│     c. Insert middle key into parent                                     │
│     d. If parent overflows, split recursively                            │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘
```

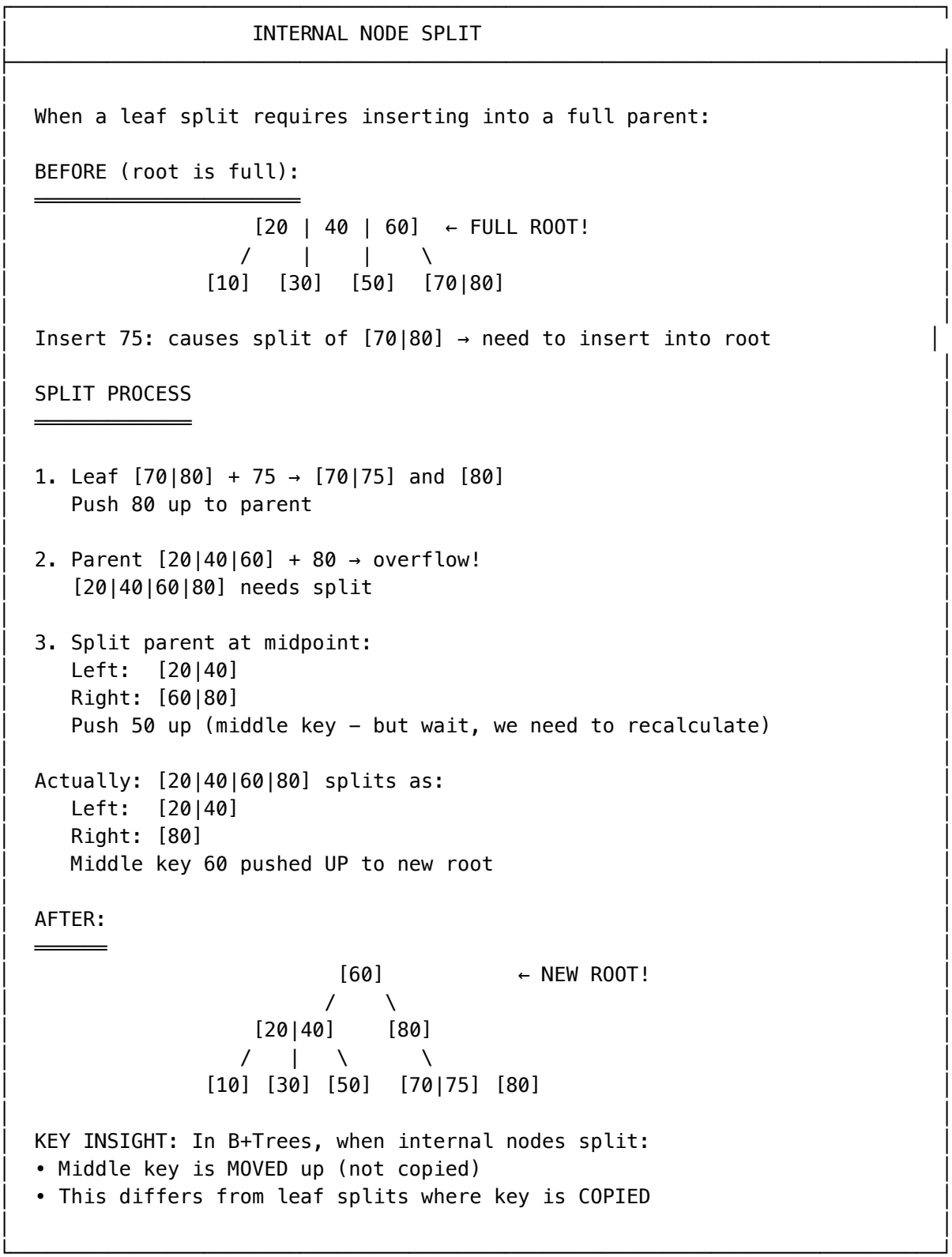
Insertion Without Split

```
┌──────────────────────────────────────────────────────────────────────────┐
│                    SIMPLE INSERTION (NO SPLIT)                           │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
│  Insert key 25 into B+Tree (order 4: max 3 keys per node)                │
│                                                                          │
│  BEFORE:                                                                 │
│  ═══════                                                                 │
│                          [30]                                            │
│                         /    \                                           │
│                  [10|20]      [40|50]                                    │
│                                                                          │
│  Step 1: Search for 25 → leaf [10|20]                                    │
│  Step 2: Leaf has space (2 keys, max 3)                                  │
│  Step 3: Insert 25 in sorted position                                    │
│                                                                          │
│  AFTER:                                                                  │
│  ══════                                                                  │
│                          [30]                                            │
│                         /    \                                           │
│                [10|20|25]    [40|50]                                     │
│                     ↑                                                    │
│                  inserted                                                │
│                                                                          │
│  No structural changes needed!                                           │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘
```

Leaf Node Split



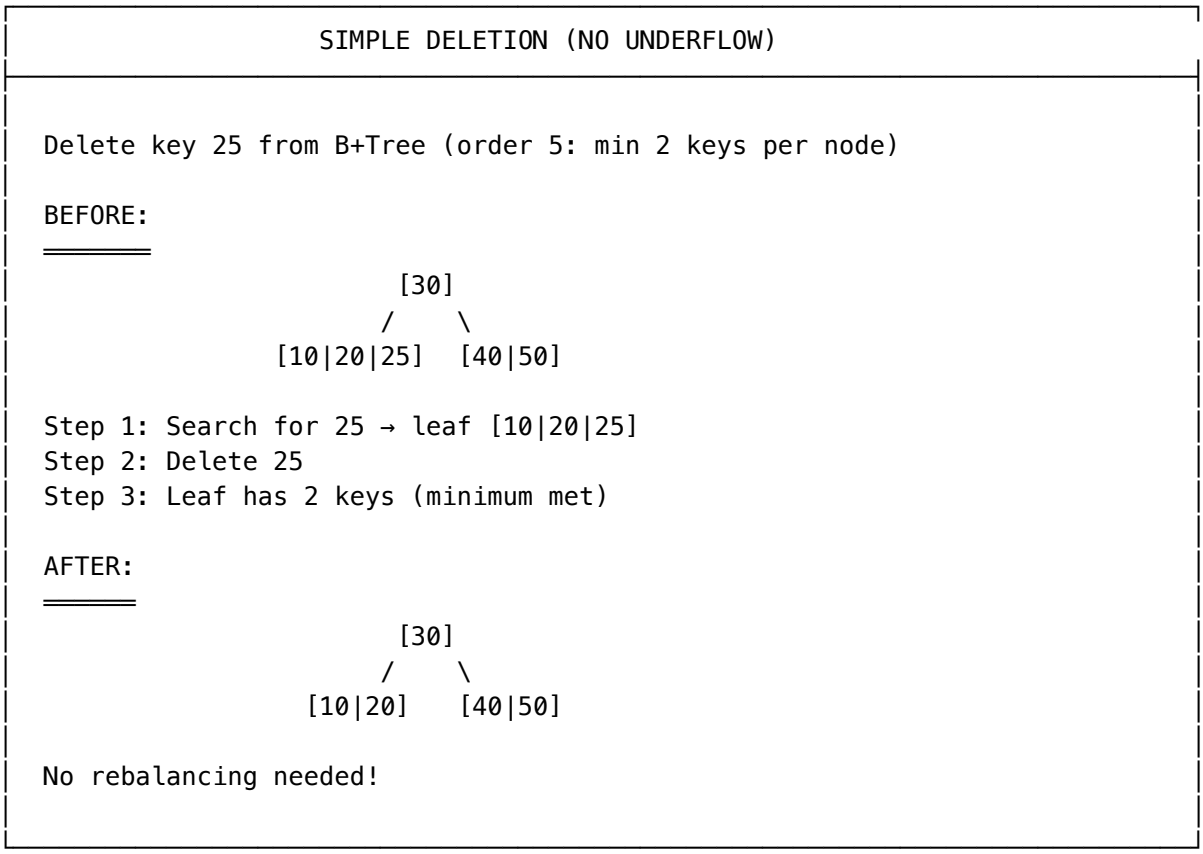
Internal Node Split (Cascading Split)



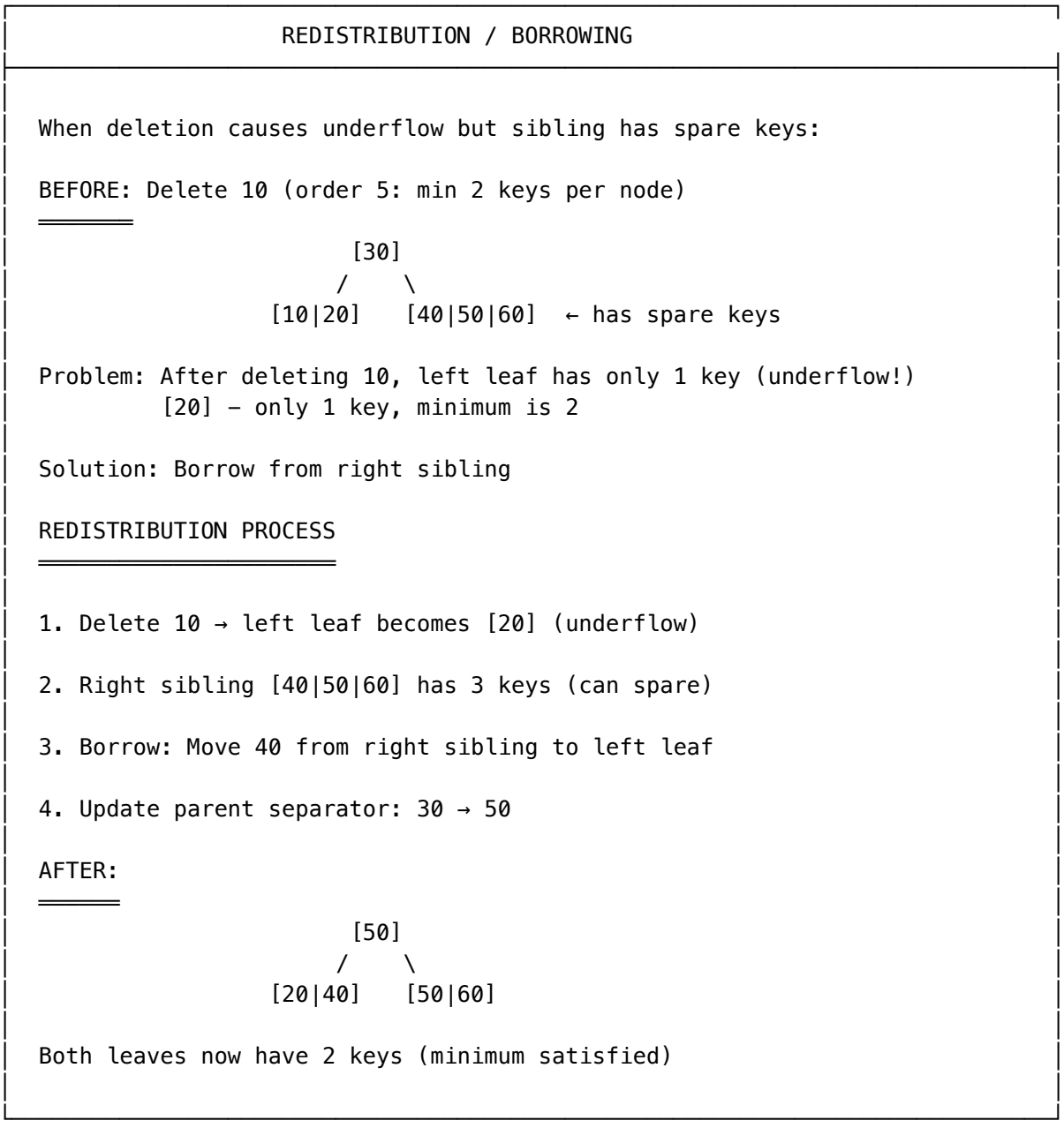
Deletion Operations

```
┌──────────────────────────────────────────────────────────────────────────┐
│                    B+TREE DELETION ALGORITHM                             │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
│  ALGORITHM: Delete(key)                                                  │
│  ═══════════════════════                                                 │
│                                                                          │
│  1. Search for key in leaf node                                          │
│  2. Delete key from leaf                                                 │
│  3. If leaf still meets minimum occupancy:                               │
│     a. Done! (update parent key if needed)                               │
│  4. If leaf underflows (below minimum):                                  │
│     a. Try borrowing from sibling                                        │
│     b. If cannot borrow, merge with sibling                              │
│     c. Update/delete parent key                                          │
│     d. If parent underflows, recurse upward                              │
│                                                                          │
│  MINIMUM OCCUPANCY                                                       │
│  ─────────────────                                                       │
│  Order M B+Tree:                                                         │
│  • Minimum keys in leaf: ⌈M/2⌉ − 1                                       │
│  • Minimum keys in internal: ⌈M/2⌉ − 1                                   │
│  • Root: no minimum (can have 1 key)                                     │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘
```

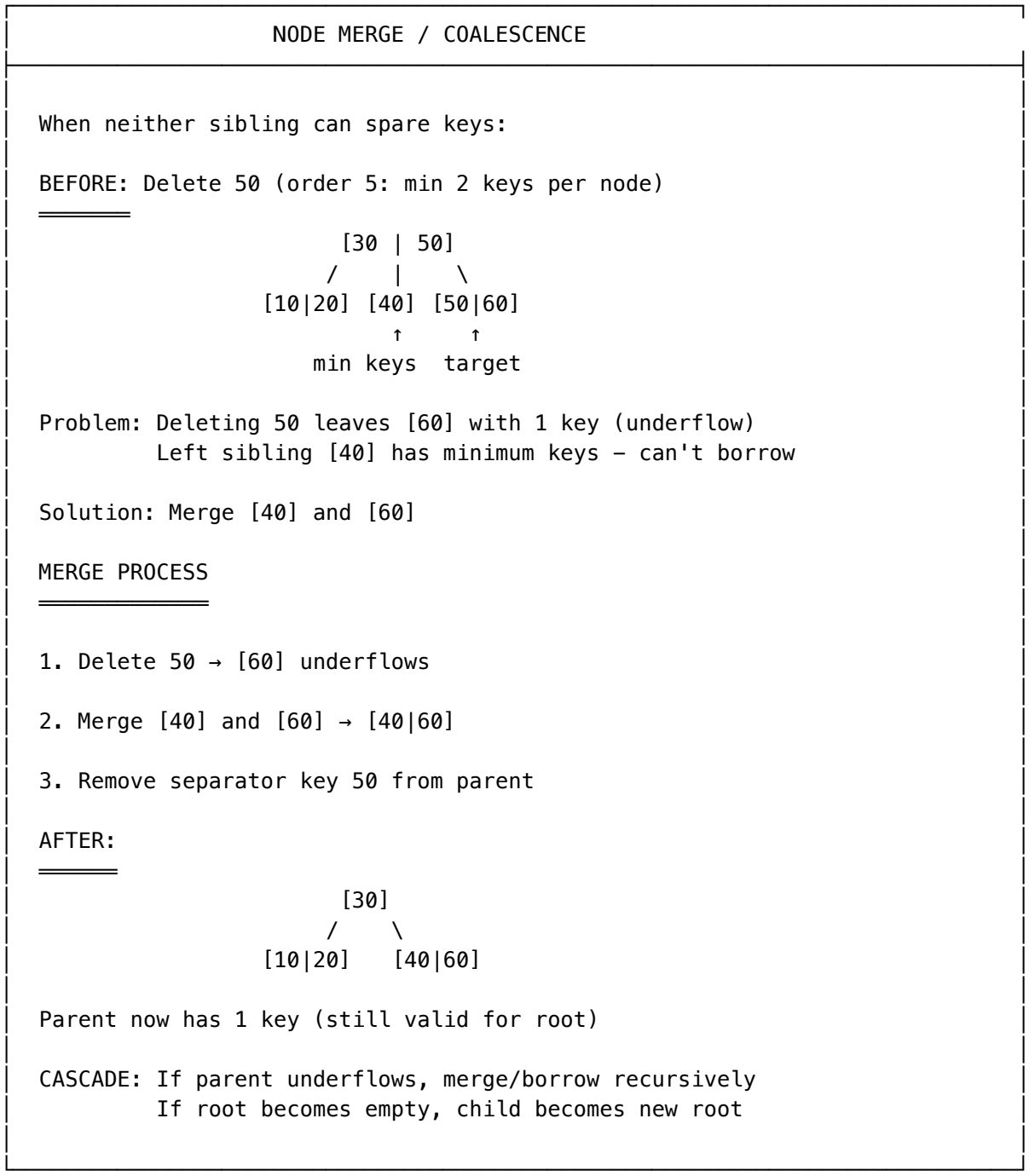
Deletion Without Underflow



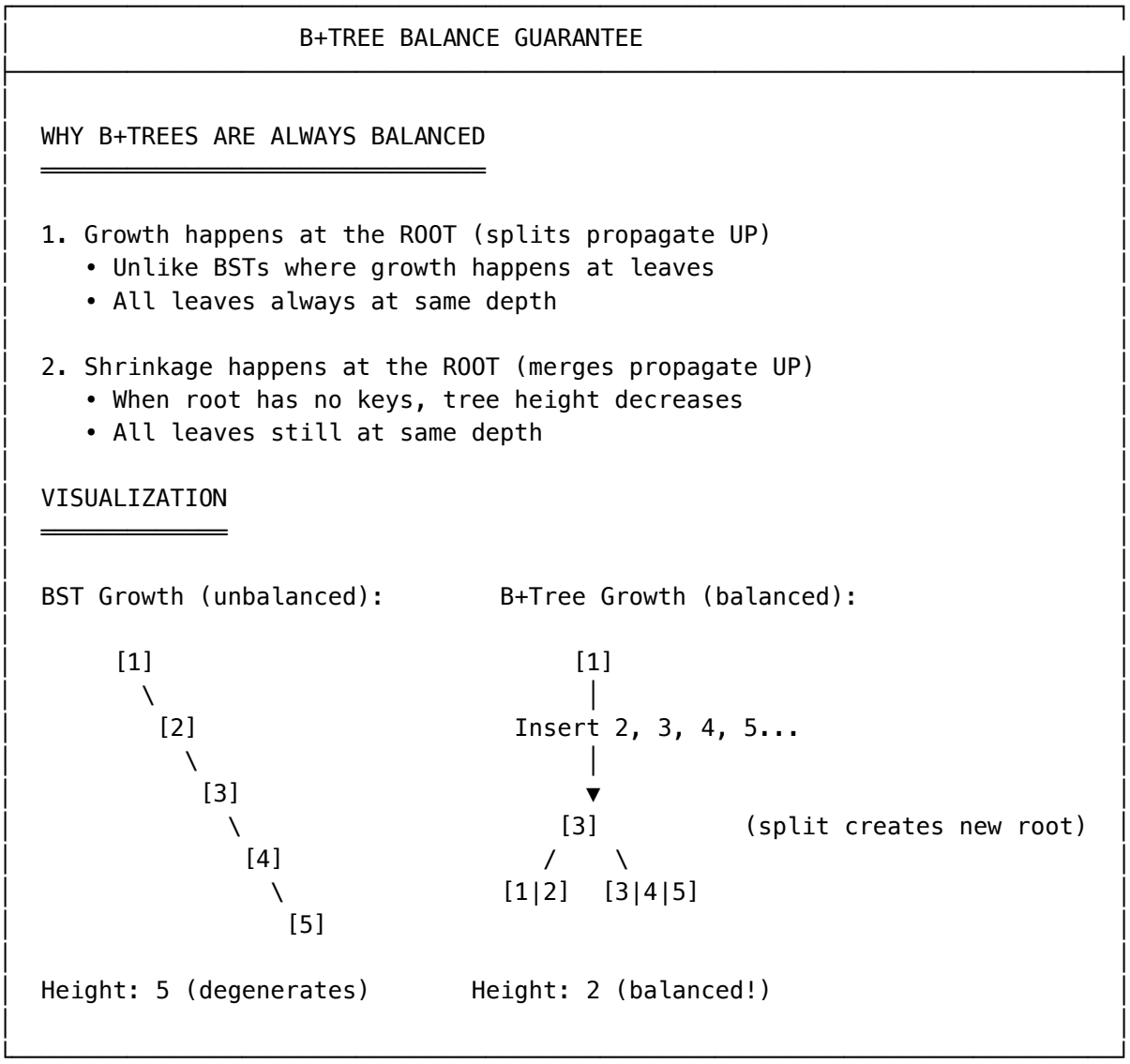
Redistribution (Borrowing from Sibling)



Merge (Coalescence)



Balancing Mechanics



Space Utilization Analysis

```
┌──────────────────────────────────────────────────────────────────────────┐
│                    SPACE UTILIZATION                                     │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
│  FILL FACTOR ANALYSIS                                                    │
│  ═══════════════════                                                     │
│                                                                          │
│  For B+Tree of order M:                                                  │
│  • Maximum keys per node: M − 1                                          │
│  • Minimum keys per node: ⌈M/2⌉ − 1                                      │
│                                                                          │
│  Minimum fill factor = (⌈M/2⌉ − 1) / (M − 1) ≈ 50%                       │
│                                                                          │
│  Example: Order 100 B+Tree                                               │
│  • Max keys: 99                                                          │
│  • Min keys: 49                                                          │
│  • Min fill: 49/99 = 49.5%                                               │
│                                                                          │
│  WORST CASE vs AVERAGE CASE                                              │
│  ══════════════════════════                                              │
│                                                                          │
│  Worst case: All nodes at minimum (50% full)                             │
│  Average case (random inserts): ~69% full                                │
│  Best case (bulk load): ~100% full (with proper algorithms)              │
│                                                                          │
│  IMPLICATIONS                                                            │
│  ════════════                                                            │
│  • Even worst case guarantees reasonable space usage                     │
│  • No need for periodic rebalancing/compaction                           │
│  • Sequential inserts may cause suboptimal fill                          │
│  • Bulk loading can achieve near-100% utilization                        │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘
```

Disk I/O Optimization

```
┌──────────────────────────────────────────────────────────────────────┐
│                  B+TREE I/O OPTIMIZATION TECHNIQUES                  │
└──────────────────────────────────────────────────────────────────────┘

1. PAGE SIZE ALIGNMENT
═══════════════════════
• Node size = disk page size (typically 4KB, 8KB, or 16KB)
• One I/O = one complete node read
• Maximizes data retrieved per disk access

Typical page sizes:
• PostgreSQL: 8 KB
• MySQL InnoDB: 16 KB
• SQLite: 4 KB (configurable)
• Oracle: 8 KB (configurable)

2. FANOUT MAXIMIZATION
═══════════════════════
• More keys per node = lower tree height = fewer I/Os
• Key compression (prefix compression, suffix truncation)
• Pointer compression (relative page numbers)

Example: 16KB page, 8-byte keys, 8-byte pointers
Fanout = 16384 / (8 + 8) ≈ 1000 children per node!

Height for 1 billion records:
log₁₀₀₀(10⁹) = 3 levels → only 3 I/Os per lookup

3. BUFFER POOL CACHING
═══════════════════════
• Keep hot pages (especially root and upper levels) in memory
• Root node: almost always cached (1 page)
• Level 1: likely cached (few hundred pages)
• Leaves: may require disk I/O

Effective I/O per lookup ≈ 1-2 (not tree height)

4. PREFETCHING
═══════════════
• For range scans, prefetch next leaf pages
• Sequential read is faster than random read
• Exploit disk read-ahead capabilities

5. BULK LOADING
═══════════════
• For initial data load, build bottom-up
• Sort data first, then create leaves
• Build internal nodes from leaf boundaries
• Achieves ~100% fill factor
```

Practical Performance Numbers

```
┌─────────────────────────────────────────────────────────────────────────┐
│                    REAL-WORLD B+TREE PERFORMANCE                        │
└─────────────────────────────────────────────────────────────────────────┘

LOOKUP LATENCY (assuming warm cache for upper levels)
═══════════════════════════════════════════════════

Records    Tree Height    Uncached I/Os    Typical Latency
────────   ────────────   ──────────────   ─────────────────
10K        2              1                ~0.1 ms (SSD)
1M         3              1-2              ~0.2 ms (SSD)
100M       3-4            1-2              ~0.3 ms (SSD)
1B         4              1-2              ~0.4 ms (SSD)

With spinning disk: multiply by 10-100x

RANGE SCAN THROUGHPUT
═════════════════════

Sequential leaf scan: ~100-500 MB/s (limited by disk/network)
Records per second: millions (depends on record size)

INSERT THROUGHPUT
═════════════════

Random inserts: 10K-50K ops/sec (depends on write strategy)
Bulk load: 100K-1M+ records/sec

KEY INSIGHT
═══════════
B+Trees are optimized for disk access patterns:
• Read a page → process many keys
• Sequential access → exploit disk prefetching
• Cache upper levels → minimize random I/O
```

Summary

```
┌──────────────────────────────────────────────────────────────────────────┐
│                    CHAPTER 2 KEY TAKEAWAYS                               │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
│  1. B-Trees exist because disk I/O is expensive                          │
│     → High branching factor minimizes tree height                        │
│     → Each disk read yields maximum useful information                   │
│                                                                          │
│  2. B+Trees are the database standard                                    │
│     → Data only in leaves (higher fanout for internal nodes)             │
│     → Linked leaves enable efficient range scans                         │
│     → Consistent lookup performance (all data at same depth)             │
│                                                                          │
│  3. Operations maintain balance automatically                            │
│     → Inserts: split nodes when full, propagate upward                   │
│     → Deletes: borrow or merge when underflow, propagate upward          │
│     → Tree grows/shrinks at the root, not leaves                         │
│                                                                          │
│  4. Performance characteristics                                          │
│     → O(log N) for all operations                                        │
│     → In practice: 3-4 I/Os for billions of records                      │
│     → With caching: often just 1-2 I/Os                                  │
│                                                                          │
│  WHAT'S NEXT                                                             │
│  ──────────                                                              │
│  Chapter 3: B-Tree Variants                                              │
│  • B*-Trees (higher fill factor)                                         │
│  • B-link Trees (better concurrency)                                     │
│  • Copy-on-Write B-Trees (MVCC support)                                  │
│  • Fractal Trees (write optimization)                                    │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘
```